```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Lasso, Ridge
from sklearn.metrics import mean_squared_error, r2_score
```

```
df = pd.read_csv("/content/CO2 Emissions_Canada.csv")

df.head()
```

	Make	Model	Vehicle Class	Engine Size(L)	Cylinders	Transmission	Fuel Type	Consum City (
0	ACURA	ILX	COMPACT	2.0	4	AS5	Z	
1	ACURA	ILX	COMPACT	2.4	4	M6	Z	
2	ACURA	ILX HYBRID	COMPACT	1.5	4	AV7	Z	
3	ACURA	MDX 4WD	SUV - SMALL	3.5	6	AS6	Z	
4	ACURA	RDX AWD	SUV - SMALL	3.5	6	AS6	Z	

Next steps: Generate code with df New interactive sheet

```
df.columns
```

```
Index(['Make', 'Model', 'Vehicle Class', 'Engine Size(L)', 'Cylinders',
       'Transmission', 'Fuel Type', 'Fuel Consumption City (L/100 km)',
       'Fuel Consumption Hwy (L/100 km)', 'Fuel Consumption Comb (L/100
km)',
       'Fuel Consumption Comb (mpg)', 'CO2 Emissions(g/km)'],
      dtype='object')
```

```
X = df[['Engine Size(L)', 'Cylinders', 'Fuel Consumption Comb (L/100 km)']]
y = df['CO2 Emissions(g/km)']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
lr = LinearRegression()
lr.fit(X_train_scaled, y_train)

y_pred_lr = lr.predict(X_test_scaled)

mse_lr = mean_squared_error(y_test, y_pred_lr)
r2_lr = r2_score(y_test, y_pred_lr)

print("Linear Regression MSE:", mse_lr)
print("Linear Regression R2:", r2_lr)
```

```
Linear Regression MSE: 421.92233190519977
Linear Regression R2: 0.8773348735033225
```

```
lasso = Lasso(alpha=0.1)
lasso.fit(X_train_scaled, y_train)

y_pred_lasso = lasso.predict(X_test_scaled)

mse_lasso = mean_squared_error(y_test, y_pred_lasso)
r2_lasso = r2_score(y_test, y_pred_lasso)

print("Lasso Regression MSE:", mse_lasso)
print("Lasso Regression R2:", r2_lasso)
```

```
Lasso Regression MSE: 421.9906734651132
Lasso Regression R2: 0.8773150046187961
```

```
ridge = Ridge()

param_grid = {'alpha': [0.01, 0.1, 1, 10, 50, 100]}

grid = GridSearchCV(ridge, param_grid, cv=5)
grid.fit(X_train_scaled, y_train)

best_ridge = grid.best_estimator_

print("Best Alpha:", grid.best_params_)
```

```
Best Alpha: {'alpha': 10}
```

```
y_pred_ridge = best_ridge.predict(X_test_scaled)

mse_ridge = mean_squared_error(y_test, y_pred_ridge)
r2_ridge = r2_score(y_test, y_pred_ridge)

print("Ridge Regression MSE:", mse_ridge)
print("Ridge Regression R2:", r2_ridge)
```

```
Ridge Regression MSE: 421.94896543141095
Ridge Regression R2: 0.8773271303605337
```

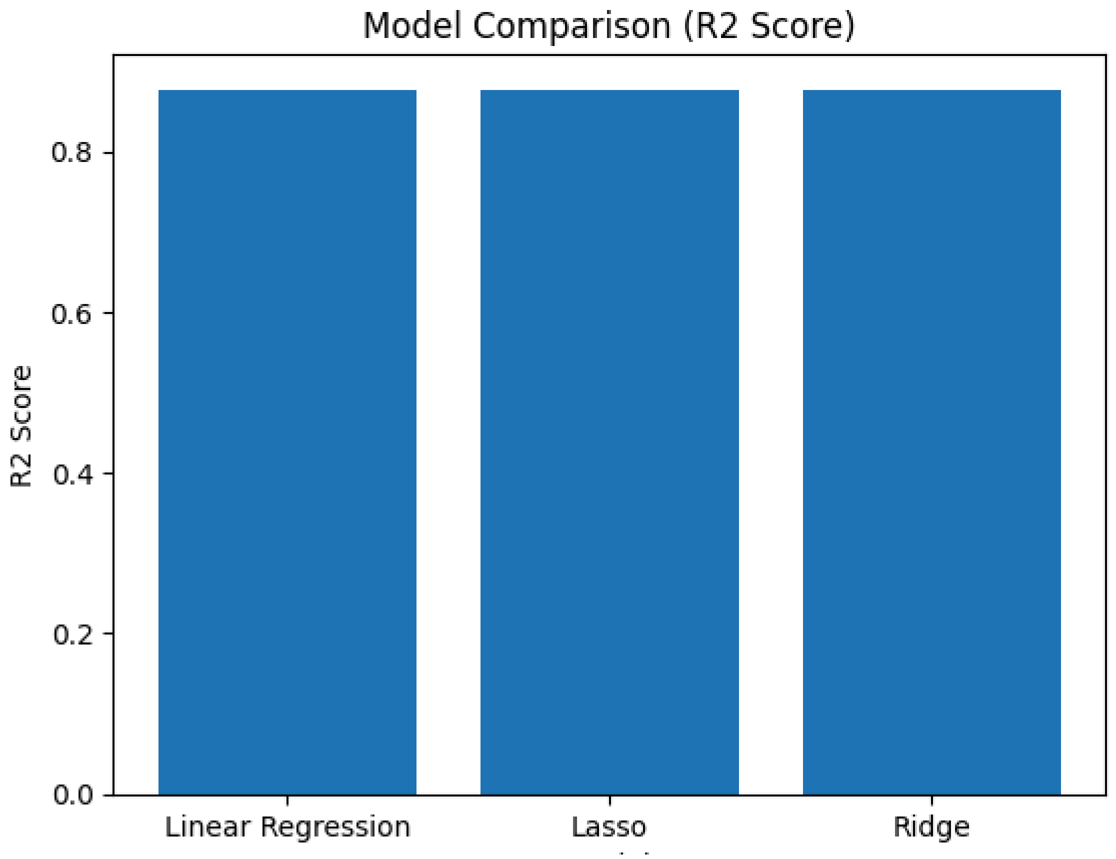
```
results = pd.DataFrame({
    "Model": ["Linear Regression", "Lasso", "Ridge"],
    "MSE": [mse_lr, mse_lasso, mse_ridge],
    "R2 Score": [r2_lr, r2_lasso, r2_ridge]
})

results
```

	Model	MSE	R2 Score
0	Linear Regression	421.922332	0.877335
1	Lasso	421.990673	0.877315
2	Ridge	421.948965	0.877327

Next steps: Generate code with results New interactive sheet

```
plt.figure()
plt.bar(results["Model"], results["R2 Score"])
plt.title("Model Comparison (R2 Score)")
plt.xlabel("Model")
plt.ylabel("R2 Score")
plt.show()
```


Model Comparison (R2 Score)
R2 Score
0.0
0.2
0.4
0.6
0.8
Linear Regression
Lasso
Ridge